

TLAMATINI

Complete Project Dossier: what the system does, how it works, how to use it, complete tracked file tree, and effective line inventory



Measure	Value
Generated	May 20, 2026 12:28 AM UTC-0600
Current HEAD	efb8c13 - Analyzer and Reviewer agents!!!, by angelahack1
Resolved version	1.4.0 (git)
Tracked files	462
Workflow agents	64
Total effective lines	100,277
Total physical text lines	136,474

1. What Tlamatini Is

- Tlamatini is a local-first AI developer assistant built with Django, Django Channels, LangChain, LangGraph, FAISS/BM25 retrieval, and a large in-repository agent application.
- It combines a browser chat surface, a Retrieval-Augmented Generation stack, a Multi-Turn tool executor, MCP-backed context providers, wrapped chat-agent runtimes, and a visual Agentic Control Panel for workflow design.
- The platform is designed for development operations: codebase analysis, file and directory context, command execution, Python execution, screenshots, web/search helpers, notifications, DevOps tools, local model operation, and Windows packaging.

What the system does

- Answers codebase questions with loaded file or directory context.
- Uses hybrid retrieval to extract metadata, split content, rank source chunks, and respect context budgets.
- Gives operators GUI-first database maintenance through the new DB dropdown for backup and staged database replacement.
- Warns GPU-host operators before a directory-context load is likely to saturate VRAM and degrade embedding throughput.
- Exposes a coherent versioning surface across builds, runtime UI, logs, and an open health-check endpoint.
- Can drive a live Unreal Engine 5 editor through the Unreal MCP plugin, from either Multi-Turn chat or the visual workflow canvas.
- Actively reaps orphaned Windows console-host and pool-child processes so long Multi-Turn or ACPX sessions do not leave misleading Tlamatini-icon ghosts in Task Manager.
- Runs checked Multi-Turn requests through request-scoped planning, capability selection, tool calls, observations, monitoring, and final synthesis.
- Launches wrapped copies of selected workflow agents in isolated runtime folders without mutating templates.
- Lets users design, validate, save, pause, resume, and stop visual workflows through the Agentic Control Panel.
- Turns successful Multi-Turn tool executions into starter `.flw`` workflows that can be inspected and validated in ACP.
- Packages the project into a distributable Windows release with installer and uninstaller tooling.

How it works

- Browser UI sends chat and workflow requests through Django views and Channels WebSockets.
- RAG chains load selected file/directory context, retrieve relevant chunks, and build answer prompts.
- DB-menu actions validate directories or SQLite files in the browser, then call Django views that either copy the live database out or stage a replacement into ``DB/ToLoad/db.sqlite3``.
- Before a heavy directory embedding run on supported NVIDIA hosts, a fail-open pre-flight guard can estimate VRAM pressure and surface a non-blocking warning in chat.
- Version resolution now flows through git tags, a runtime resolver module, generated build artefacts, and an open ``/agent/version/`` endpoint.
- When Multi-Turn is enabled, the global planner selects context and tool stages before the executor binds only the relevant tools, including wrapped deterministic agents such as De-Compressor.
- The Unreal path opens a TCP socket to the Unreal MCP plugin, sends one ``{"type": "command", "params": {...}}`` payload, captures the JSON reply, and emits one ``INI_SECTION_UNREALER`` block for downstream logic.
- After spawn-capable tool calls and again after the final answer, the orphan reaper can sweep dead descendants, orphaned ``conhost.exe`` companions, and stale pool-linked processes without ever raising into the chat path.
- Tool calls execute in the backend, append observations, and may create wrapped runtime copies under ``agent/agents/pools/_chat_runs_``.
- On the next full start-up, ``manage.py`` can swap a staged database into place before Django imports, while archiving the previous live database under ``DB/Older/<timestamp>``.

- ACP flows deploy session-scoped pool instances, wire config values, validate NxN graph rules, and execute through Starter-driven flow semantics.
- Build scripts collect static assets, bundle Django/Python resources, add agent templates, and assemble `pkg.zip`, `Uninstaller.exe`, and `dist/Tlamatini\_Release/`.

2. Architecture Layers

Layer	Role
Browser interfaces	Chat page plus Agentic Control Panel templates and JavaScript modules.
Django/Channels	Authentication, views, WebSockets, session state, message persistence, and ASGI startup.
RAG and context	Metadata extraction, text splitting, FAISS/BM25 retrieval, context budgeting, and fallback behavior.
Multi-Turn engine	Capability registry, global execution planner, explicit tool loop, answer parsing, and answer-success classification.
Tools and agents	Core tools, MCP context providers, wrapped chat-agent launchers, and the current visual workflow agent templates.
Packaging	PyInstaller build scripts, shortcut registration, `.flw` association, installer, uninstaller, and release folder assembly.

Design principles

- Evidence-first answers: Tlamatini grounds responses in selected project context and hybrid retrieval rather than freeform model memory.
- Explicit orchestration: checked Multi-Turn uses a visible tool loop, capability scoring, and staged planning instead of a single opaque call.
- Operational reversibility: risky changes such as database replacement are staged, archived, and delayed to the only safe window instead of hot-swapped mid-session.
- Fail-open diagnostics: GPU pressure probes and session-restore safeguards warn early without breaking CPU-only or degraded environments.
- Runtime isolation: wrapped chat-agent copies run in session-scoped folders so template agents remain pristine while live runs stay inspectable.
- Operator truth over vibes: Exec Report tables, tlamatini.log, skill audits, and ACPX transcripts make the system auditable after execution.

3. Installation, Configuration, and Everyday Use

Installation essentials

- Python 3.12.10 is the strongly recommended source-mode version in the README, and the codebase has been tested most deeply there.
- Source installs require a clone, virtual environment, dependency install from `requirements.txt`, migrations, a superuser, static collection, and then the web server.
- You can run either the checked-in cloud/back-end defaults from `Tlmatini/agent/config.json` or a local Ollama-backed configuration with matching model names.
- Packaged Windows installs create a default `user / changeme` account; manual source installs use your own `createsuperuser` account instead.

Configuration essentials

- Source mode resolves `Tlmatini/agent/config.json`; frozen builds resolve `config.json` next to the executable; `CONFIG\_PATH` overrides both.
- Core keys include `embedding-model`, `chained-model`, `ollama\_base\_url`, `ollama\_token`, `enable\_unified\_agent`, `unified\_agent\_model`, and `unified\_agent\_max\_iterations`.
- The chat-side Config -> Models and Config -> URLs dialogs are now first-class configuration surfaces, and they can explicitly ask the operator to reconnect when saved values change live-session assumptions.
- The separate DB dropdown is not a config editor: it is a maintenance surface for copying the live SQLite database out or staging a replacement for the next full start-up.
- Multi-Turn is toggled from the chat toolbar, but it depends on the unified-agent configuration and the selected model/base-url pairing being valid.
- Image interpretation can run through Claude-backed cloud paths or Qwen/Ollama-backed local paths, and remote Ollama can be protected with a bearer token.

DB menu and database swap-in

- The new DB dropdown gives operators two GUI-first maintenance paths: `Backup database` copies the live SQLite file out, and `Set DB` stages a chosen `db.sqlite3` for the next full start-up.
- Both dialogs are live-validated in the browser and now expose Browse buttons that open native host-side pickers for folders or `db.sqlite3` files.
- Backup is read-only and uses the currently live database path; Set DB never hot-swaps the file mid-session because Django already holds SQLite open.
- Set DB writes the selected file to `DB/ToLoad/db.sqlite3`; the real swap happens only at the top of `manage.py` before Django imports anything.
- When that swap runs, the previous live database is moved into `DB/Older/<timestamp>/db.sqlite3`, creating a built-in rollback trail instead of overwriting history.
- Reconnect is not enough for this path: the operator must fully restart Tlmatini so the pre-Django swap window opens again.
- If the staged file is bad or locked, startup fails open: Tlmatini logs the error and continues with the previous live database.

Versioning system

- Tlmatini now follows Semantic Versioning 2.0.0 with git tags as the single source of truth: you tag, then you build, instead of hand-editing version strings across files.
- The build path resolves a version once and propagates it into generated runtime metadata, Win32 VERSIONINFO resources, and the release-folder naming convention.
- On untagged commits the resolver falls back honestly to a git-derived development version instead of failing the build or lying about the release state.

Version surfaces

- Operators can now see the running version in the About dialog, the startup banner, the open `GET /agent/version/` health-check endpoint, and Windows file properties on the built executables.
- The runtime resolver lives in `Tlmatini/agent/version.py`, while the build-oriented coordination logic lives in the repo-root `versioning.py` and the longer policy notes live in `VERSIONING.md`.
- Build overrides are supported in a predictable order: CLI `--version`, then `TLMATINI_VERSION`, then `git describe`, then the sentinel `0.0.0+unknown`.

De-Compressor agent

- De-Compressor is the deterministic archive worker for compression and decompression tasks, deciding direction from whichever side exposes a recognized archive extension.
- Supported archive families are `.gz`, `.zip`, `.7z`, `.tar.gz`, and `.gz.tar`; file-to-folder extraction and file-or-directory packing are both documented in the README and Book.
- Password handling is explicit: `passwordless=true` skips it, while `passwordless=false` requires the `DE_COMPRESSER_PWD` environment variable and fails fast if the secret is missing.

De-Compressor integration and fallback behavior

- The agent is reachable from both operator surfaces: ACP canvas nodes wire through dedicated connection-update views, and checked Multi-Turn can invoke it through `chat_agent_de_compressor`.
- Format engines are practical rather than magical: `stdlib`gzip`` and `zipfile`` cover core cases, `7z`` is preferred for encrypted `7z`` or `zip``, and `py7zr`` was added to `requirements.txt`` as the Python fallback.
- Every run emits an `INI_SECTION_DE_COMPRESSER<<< ... >>>END_SECTION_DE_COMPRESSER`` block and still triggers `target_agents``, so downstream Parametrizer or Raiser logic can branch on `success=true|false`` instead of guessing from prose.

Unreal MCP and the Unreal agent

- Unreal MCP is a UE5 plugin that runs inside the editor and listens on `127.0.0.1:55557`` for one JSON command per TCP connection; Tlmatini is the client side of that link, not the plugin host.
- The Unreal workflow agent exposes the upstream 28-command surface: actor manipulation, Blueprint authoring and node wiring, input mappings, and UMG widget building.
- The same integration is available in both operator surfaces: checked Multi-Turn via `chat_agent_unreal``, and ACP canvas flows via the visual Unreal node plus Parametrizer chaining.

Installing the UE5 plugin and smoke-testing it

- Install the upstream plugin into `<YourProject>/Plugins/UnrealMCP/``, enable it from `Edit -> Plugins``, restart UE5, and confirm the Output Log says `UnrealMCP listening on 127.0.0.1:55557``.
- Tlmatini does not compile or embed the plugin; the Unreal editor must already be running with the listener bound before any Unreal call can succeed.
- A seeded smoke test ships in the Prompts table: `Unreal MCP End-to-End Editor Drive`` walks through sanity-check, actor spawn, Blueprint creation/compile, and UMG widget assembly.

Orphan-process cleanup

- Tlmatini now ships a three-tier orphan reaper in `Tlmatini/agent/orphan_reaper.py`` focused on Windows console-host leftovers such as `conhost.exe`` and `openconsole.exe``.
- Tier 1 runs after spawn-capable Multi-Turn tool calls, Tier 2 runs once after the final answer in a background thread, and Tier 3 runs again during application shutdown through the same cleanup path that already tears down pools.
- The candidate set stays narrow on purpose: dead descendants of the current process tree, orphaned console hosts tied to that tree, and pool-linked processes whose `cmdline`` still points into `agents/pools/...`` even though tracking records are gone.

Orphan-process prevention and survivor reporting

- Prevention landed alongside cleanup: Windows spawn sites now use ``CREATE_NO_WINDOW | DETACHED_PROCESS | CREATE_NEW_PROCESS_GROUP`` with stdio piped to ``DEVNULL``, so the console host is usually never allocated in the first place.
- The ACPX runtime now kills full process trees instead of only top-level wrappers, and pool-agent scripts gained a conservative ``subprocess.Popen`` guard so forgotten descendants still inherit ``CREATE_NO_WINDOW``.
- If something survives both cleanup tiers, Tlmatini sends a second chat bubble listing each surviving ``name + PID``; the reaper never raises into the user path because a cleanup crash would be worse than the leftovers it tried to remove.

How to use it

- Run from source: create a virtual environment, install requirements, migrate, create a superuser, collect static files, and start Django.
- Open ``/agent/`` for chat. Load a file or directory context before asking codebase-specific questions.
- Keep Multi-Turn unchecked for direct Q&A; enable Multi-Turn for tasks that need tools, wrapped agents, monitoring, or workflow seeding.
- For Unreal Engine work, enable the Unreal MCP plugin inside a live UE5 project first, then call ``chat_agent_unrealer`` from Multi-Turn or use the visual Unreal node on the canvas.
- Archive jobs can now be described directly in Multi-Turn or modeled visually in ACP: De-Compressor infers compress vs decompress from the ``input`` or ``output`` extension.
- If a second post-answer warning bubble ever lists surviving ``name + PID`` entries, treat it as an honest cleanup report and end the listed processes manually from Task Manager if needed.
- Use the ``ACPX-Skills`` navbar menu when you need to browse, enable/disable, diagnose, or reload the shipped SKILL.md catalog without asking the LLM to be your admin surface.
- Use the DB dropdown when you need a safe database snapshot or want to stage a different ``db.sqlite3`` for the next start-up without hot-swapping the live SQLite file.
- Open ``/agentic_control_panel/`` to drag agents, connect them, configure each node, validate, start, pause/resume, stop, and save ``.flw`` workflows.
- Use ``python build.py``, ``python build_uninstaller.py``, and ``python build_installer.py`` only when producing a packaged Windows release.

Agent descriptions and catalog source of truth

- The authoritative human-readable source for workflow-agent descriptions is ``agents_descriptions.md`` at the repo root, not an embedded JavaScript map or a hard-coded Django list.
- Current validation compares live template directories, ``agents_descriptions.md`` rows, and the README / Book bestiary counts so the generated dossier stays aligned with the running product.
- The ACP sidebar hover tooltip and the right-click Description dialog both resolve through that markdown file first; ``README.md`` is only a legacy fallback when the dedicated description file is absent.

Reviewer and Analyzer

- The workflow catalog now includes two new high-value specialists: Reviewer and Analyzer, lifting the canvas inventory to 64 agents and the seed-skill catalog to 23 SKILL.md packages.
- Reviewer is LLM-powered: it resolves a git diff for ``repo_path``, reviews it with a senior-engineer prompt, and emits an ``INI_SECTION_REVIEWER`` block whose first routable field is ``verdict = APPROVE | REQUEST_CHANGES | COMMENT``.
- Analyzer is deterministic and scanner-driven: it runs whichever of ``bandit``, ``semgrep``, ``ruff``, ``eslint``, ``gitleaks``, and ``pip-audit`` are installed on PATH over ``target_path``, then emits an ``INI_SECTION_ANALYZER`` block with ``status`` and ``total_findings`` for downstream routing.
- Both agents always trigger ``target_agents``, so a downstream Forker can branch on ``{verdict}``, ``{status}``, or ``{total_findings}`` instead of scraping prose.

- They are intentionally canvas-only workflow agents: there is no wrapped `chat\_agent\_reviewer` or `chat\_agent\_analyzer`, and therefore no duplicate Exec Report row family for those names.
- The chat-side counterparts live in the ACPX skill catalog instead: `code-review` exposes senior-engineer git-diff review, while `security-audit` exposes the deterministic multi-scanner sweep.

ACPX-Skills menu

- The new `ACPX-Skills` navbar menu gives operators four direct actions over the skill catalog: Browse Skills, Configure Skills, Diagnostics, and Reload Registry.
- `Configure Skills` flips `Skill.enabled` exactly the way MCPs and Tools are toggled, so disabled skills disappear from `list\_skills` and reject `invoke\_skill` with `SKILL\_DISABLED` instead of silently half-working.
- The diagnostics view cross-checks skill dependencies against disabled tools, disabled MCPs, missing ACPX agents, and orphan database rows whose SKILL.md disappeared from disk.

Source-mode bootstrap commands

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python Tlamatini/manage.py migrate
python Tlamatini/manage.py createsuperuser
python Tlamatini/manage.py collectstatic --noinput
python Tlamatini/manage.py runserver --noreload
```


4. Ollama Setup Without Administrative Rights

- Open a normal PowerShell window, not an elevated one, for the safest no-admin Windows installation path.
- Install into `%LOCALAPPDATA%\Programs\Ollama` with the official PowerShell installer script and then reopen PowerShell so PATH updates are visible.
- Verify the CLI with `ollama --version`, start `ollama serve` if the background service is not already active, and confirm `http://127.0.0.1:11434/api/tags` responds.
- Pull the default repository model tags exactly as written if you want the shipped config and agent templates to work unchanged.

No-admin Ollama commands and default model pulls

```
$env:OLLAMA_INSTALL_DIR = "$env:LOCALAPPDATA\Programs\ollama"
irm https://ollama.com/install.ps1 | iex
ollama --version
ollama serve
Invoke-WebRequest http://127.0.0.1:11434/api/tags -UseBasicParsing
ollama pull qwen3-embedding:8b
ollama pull glm-5:cloud
ollama pull qwen3.5:cloud
ollama pull gpt-oss:120b-cloud
ollama pull qwen3.5:397b-cloud
ollama pull llama3.2-vision:11b
```

5. Runtime, Release, and Operator Diagnostics

Running the application

- Development server: ``python Tlamatini/manage.py runserver --noreload``.
- Preferred async/dev bootstrap: ``python Tlamatini/manage.py startserver``, which starts MCP services before the Django server.
- Production-style ASGI endpoint: ``daphne -b 127.0.0.1 -p 8000 tlamatini.asgi:application``.
- Current startup also re-applies GPU-performance / Ollama-pinning hooks in the background on supported NVIDIA Windows hosts, so restart-time behavior stays closer to the tuned development baseline.
- That same early startup window is also where a staged ``DB/ToLoad/db.sqlite3`` file is promoted into the live database path, before Django opens SQLite.
- Startup now also prints a ``--- [VERSION] Tlamatini ...`` banner, making the running build visible in both the console and ``tlamatini.log`` without an HTTP call.
- Startup cleans pool state, repopulates the agent registry, launches MCP metrics/file-search servers, and then serves HTTP plus WebSocket traffic.

Embedding-memory pre-flight guard

- README and BookOfTlamatini now document an embedding-memory pre-flight guard for GPU hosts before a directory-context load starts its FAISS embedding burst.
- On supported NVIDIA hosts it estimates embedding-model VRAM pressure, and when the projected load is too high it emits a non-blocking warning chat bubble instead of silently letting RAM<->VRAM thrash surprise the operator.
- CPU-only, AMD, and Apple-Silicon hosts stay fail-open: the guard becomes a no-op when the NVIDIA probe does not apply.
- The practical operator response is straightforward: switch to a smaller embedding model, reconnect if needed, or proceed knowingly with the heavier model.

Reconnect and restart safeguards

- Config -> Models and Config -> URLs dialogs now track their pre-edit baseline and can show a reconnect-required dialog when the saved values change what the live chat session should trust.
- The restored-session autoload path now buffers early WebSocket frames so context-loading spinners and disabled-input state are not lost during automatic reconnect/restore flows.
- Startup and restart behavior now also re-apply GPU performance and Ollama keep-alive hooks in the background on supported NVIDIA Windows hosts, improving warm-model readiness without blocking Django boot.
- Windows process hygiene is now part of that safety story too: detached no-window spawns and the three-tier orphan reaper reduce the chance that Task Manager shows stale Tlamatini-icon console helpers after long runs.

Prompt catalog and answer readability discipline

- Version ``1.3.2`` tightened the HTML answer contract with a Prime Directive on visual readability: explicit background and text color, no grey-on-dark body text, and safer table-body defaults.
- The seeded ``Prompts`` dropdown was also re-sorted into a learner path: context-only Q&A first, then metrics, files search, shell, code generation, vision, specialized single-tool actions, agent control, Unrealer, and heavier Multi-Turn/ACPX demos last.
- Those readability rules remain in force in the current documentation set, and the newer ``v1.4.0`` release state keeps the version badge, runtime surfaces, and operator handbook aligned.

Unreal MCP runtime behavior

- Each Unrealer run is one command: load ``config.yaml``, open a fresh TCP socket, send ``{"type": "command", "params": "params"}``, read until valid JSON arrives, and log one atomic ``INI_SECTION_UNREALER<<< ... >>>END_SECTION_UNREALER`` block.

- The wrapped-tool path stores chat runs under ``agent/agents/pools/_chat_runs/_unrealer_<seq>_<id>/'`, while visual flows use normal ``unrealer_<n>`` pool folders and can chain response fields through Parametrizer mappings.
- Exec Report treats Unrealer as its own row family, and troubleshooting stays concrete: connection-refused means the plugin is not listening, while read-timeout usually means UE5's game thread is busy.

Orphan reaper runtime behavior

- Tlmatini now ships a three-tier orphan reaper in ``Tlmatini/agent/orphan_reaper.py`` focused on Windows console-host leftovers such as ``conhost.exe`` and ``openconsole.exe``.
- Tier 1 runs after spawn-capable Multi-Turn tool calls, Tier 2 runs once after the final answer in a background thread, and Tier 3 runs again during application shutdown through the same cleanup path that already tears down pools.
- The candidate set stays narrow on purpose: dead descendants of the current process tree, orphaned console hosts tied to that tree, and pool-linked processes whose ``cmdline`` still points into ``agents/pools/...`` even though tracking records are gone.
- Prevention landed alongside cleanup: Windows spawn sites now use ``CREATE_NO_WINDOW | DETACHED_PROCESS | CREATE_NEW_PROCESS_GROUP`` with stdio piped to ``DEVNULL``, so the console host is usually never allocated in the first place.
- The ACPX runtime now kills full process trees instead of only top-level wrappers, and pool-agent scripts gained a conservative ``subprocess.Popen`` guard so forgotten descendants still inherit ``CREATE_NO_WINDOW``.
- If something survives both cleanup tiers, Tlmatini sends a second chat bubble listing each surviving ``name + PID``; the reaper never raises into the user path because a cleanup crash would be worse than the leftovers it tried to remove.

Release pipeline

- Release production is a three-step pipeline: ``build.py`` -> ``build_uninstaller.py`` -> ``build_installer.py``.
- The final distributable is the full ``dist/Tlmatini_Release/`` folder, not a stray executable copied outside its payload.
- Current ``build.py`` treats ``README.md`` and ``jd-cli/`` as required post-build assets and fails hard if those payloads are missing.
- Bundled support scripts cover shortcut creation/removal, ``.flw`` association, the PowerShell launcher, and Windows-specific installer ergonomics.

Exec Report

- Exec Report is a Multi-Turn-only transparency layer that appends one operation table per state-changing agent family to the final answer.
- Rows are recorded from the live tool-call stream rather than guessed from the LLM prose, so the report is the operational ground truth.
- Each row receives a SUCCESS/FAILURE verdict from raw tool returns, making long installs, deployments, and remediations inspectable after the fact.

ACPX and skills

- ACPX lets Tlmatini spawn external coding-agent CLIs such as Codex, Claude Code, Cursor, Gemini, Qwen, and others as managed child processes.
- It pairs those agents with markdown-driven ``SKILL.md`` packages, validated I/O contracts, permission gating, and append-only audit logs.
- Transport-aware drain rules and bounded event bodies reduce latency while protecting the LLM context budget during external-agent relays.
- The operator-facing references are ``README.md`` and ``ACPX.md``, while the implementation lives under ``agent/acpx/``, ``agent/skills/``, and ``agent/skills_pkg/``.

Application log

- The built-in ``tlmatini.log`` file captures both stdout and stderr through a tee stream initialized in ``manage.py`` before Django starts.

- In source mode the log sits next to `manage.py`; in frozen mode it lives next to the executable.
- Immediate flush behavior makes the log the primary forensic artifact for startup problems, warnings, tracebacks, and runtime diagnostics.

6. Agent Catalog and Runtime Model

Tlmatini currently exposes 64 workflow-agent templates.

Category	Representative agents
Control	starter, ender, stopper, cleaner, barrier, flowbacker
Execution and files	executer, pythonxer, pser, file_creator, file_extractor, file_interpreter, de_compressor, unrealer, mover, deleter
DevOps and infra	gitter, dockerer, kubernetes, jenkinser, ssher, scper
Data and APIs	sqler, mongoxer, apirer, crawler, googler
Monitoring and routing	monitor_log, monitor_netstat, flowhypervisor, forker, asker, counter, and, or
Communication	notifier, emailer, recmailer, telegramer, telegramrx, teletlmatini, whatsapper, whatstlmatini
Security and media	kyber_keygen, kyber_cipher, kyber_decipher, image_interpreter, shoter, j_decompiler
Workflow intelligence	flowcreator, gatewayer, gateway_relayer, node_manager, parametrizer, prompter, summarizer, acpxer

All workflow agents follow a common deployment pattern: template directory, YAML configuration, session-scoped pool copy, PID/status/log files, target/source wiring, and optional reanimation state.

Agent catalog validation

- The authoritative human-readable source for workflow-agent descriptions is `agents\_descriptions.md` at the repo root, not an embedded JavaScript map or a hard-coded Django list.
- Current validation compares live template directories, `agents\_descriptions.md` rows, and the README / Book bestiary counts so the generated dossier stays aligned with the running product.
- The ACP sidebar hover tooltip and the right-click Description dialog both resolve through that markdown file first; `README.md` is only a legacy fallback when the dedicated description file is absent.

How agent runtimes are shaped

- Every workflow agent follows the same operational skeleton: template directory, `config.yaml`, a session-scoped pool copy, PID/status/log files, and explicit source/target wiring.
- Chat-wrapped tool calls launch isolated runtime copies under `agent/agents/pools/\_chat\_runs\_/\_`, while ACP uses named pool folders such as `starter\_1` or `unrealer\_1`.
- Specialized agents now stretch the platform in different directions: ACPXer drives external coding-agent CLIs, Unrealer drives a live UE5 editor, and TeleTlmatini / WhatsTlmatini bridge full Tlmatini conversations into messaging platforms.

Reviewer and Analyzer spotlight

- The workflow catalog now includes two new high-value specialists: Reviewer and Analyzer, lifting the canvas inventory to 64 agents and the seed-skill catalog to 23 SKILL.md packages.
- Reviewer is LLM-powered: it resolves a git diff for `repo\_path`, reviews it with a senior-engineer prompt, and emits an `INI\_SECTION\_REVIEWER` block whose first routable field is `verdict = APPROVE | REQUEST\_CHANGES | COMMENT`.
- Analyzer is deterministic and scanner-driven: it runs whichever of `bandit`, `semgrep`, `ruff`, `eslint`, `gitleaks`, and `pip-audit` are installed on PATH over `target\_path`, then emits an `INI\_SECTION\_ANALYZER` block with `status` and `total\_findings` for downstream routing.
- Both agents always trigger `target\_agents`, so a downstream Forker can branch on `{verdict}`, `{status}`, or `{total\_findings}` instead of scraping prose.
- They are intentionally canvas-only workflow agents: there is no wrapped `chat\_agent\_reviewer` or `chat\_agent\_analyzer`, and therefore no duplicate Exec Report row family for those names.
- The chat-side counterparts live in the ACPX skill catalog instead: `code-review` exposes senior-engineer git-diff review, while `security-audit` exposes the deterministic multi-scanner sweep.

Unrealer spotlight

- Unreal MCP is a UE5 plugin that runs inside the editor and listens on `127.0.0.1:55557` for one JSON command per TCP connection; Tlamatini is the client side of that link, not the plugin host.
- The Unreal workflow agent exposes the upstream 28-command surface: actor manipulation, Blueprint authoring and node wiring, input mappings, and UMG widget building.
- The same integration is available in both operator surfaces: checked Multi-Turn via `chat_agent_unrealer`, and ACP canvas flows via the visual Unreal node plus Parametrizer chaining.
- Each Unreal run is one command: load `config.yaml`, open a fresh TCP socket, send `{"type": "command", "params": { "params": { "INI_SECTION_UNREALER": "..." } } }`, read until valid JSON arrives, and log one atomic `INI_SECTION_UNREALER<<< ... >>>END_SECTION_UNREALER` block.
- The wrapped-tool path stores chat runs under `agent/agents/pools/_chat_runs/_unrealer_<seq>_<id>`, while visual flows use normal `_unrealer_<n>` pool folders and can chain response fields through Parametrizer mappings.
- Exec Report treats Unreal as its own row family, and troubleshooting stays concrete: connection-refused means the plugin is not listening, while read-timeout usually means UE5's game thread is busy.

De-Compressor spotlight

- De-Compressor is the deterministic archive worker for compression and decompression tasks, deciding direction from whichever side exposes a recognized archive extension.
- Supported archive families are `.gz`, `.zip`, `.7z`, `.tar.gz`, and `.gz.tar`; file-to-folder extraction and file-or-directory packing are both documented in the README and Book.
- Password handling is explicit: `passwordless=true` skips it, while `passwordless=false` requires the `DE_COMPRESSER_PWD` environment variable and fails fast if the secret is missing.
- The agent is reachable from both operator surfaces: ACP canvas nodes wire through dedicated connection-update views, and checked Multi-Turn can invoke it through `chat_agent_de_compressor`.
- Format engines are practical rather than magical: `stdlib`gzip`` and `zipfile`` cover core cases, `7z`` is preferred for encrypted `7z`` or `zip``, and `py7zr`` was added to `requirements.txt`` as the Python fallback.
- Every run emits an `INI_SECTION_DE_COMPRESSER<<< ... >>>END_SECTION_DE_COMPRESSER` block and still triggers `target_agents`, so downstream Parametrizer or Raiser logic can branch on `success=true|false`` instead of guessing from prose.

Orphan-reaper spotlight

- Tlamatini now ships a three-tier orphan reaper in `Tlamatini/agent/orphan_reaper.py`` focused on Windows console-host leftovers such as `conhost.exe`` and `openconsole.exe``.
- Tier 1 runs after spawn-capable Multi-Turn tool calls, Tier 2 runs once after the final answer in a background thread, and Tier 3 runs again during application shutdown through the same cleanup path that already tears down pools.
- The candidate set stays narrow on purpose: dead descendants of the current process tree, orphaned console hosts tied to that tree, and pool-linked processes whose `cmdline`` still points into `agents/pools/...`` even though tracking records are gone.

7. Repository Facts and Git Changes

Metric	Value
Tracked files in git	462
Workflow agents	64
agents_descriptions.md rows	64
Django migrations	89
Frontend JavaScript modules	28
Frontend CSS files	6
HTML templates	4
Python requirements	63
Binary/asset tracked files skipped from line count	15
Resolved version	1.4.0
Version source	git

Latest commits

Date	Commit	Subject
2026-05-20	efb8c13	Analyzer and Reviewer agents!!!, by angelahack1
2026-05-19	bd0494c	Updating documentation, by Tlamatini's-AutoBot.
2026-05-19	27366ea	Updating documentation according to version, by Tlamatini's-AutoBot.
2026-05-19	141d104	Sorting catalog of prompts, and instructing LLM to generate content very visible and accesible, not using similar colors for text and i
2026-05-18	fae830c	Fixing prompt repeated, by Tlamatini's-AutoBot.
2026-05-18	9392af4	Intuitively sorting prompt, and directives to LLM to use Executer and Pythonxer instead of Keyboarder/Mouser in making scripts fo
2026-05-18	6fcc13f	Bringing logo upwards, by Tlamatini's-AutoBot.
2026-05-18	d07f956	Adding header to README.md, by Tlamatini's-AutoBot.
2026-05-18	0aa5c52	Updating version in all the documentation and installing/uninstalling windows, by angelahack1
2026-05-18	2c707e3	Patching Ruff availability all along Multi-turn execution., by angelahack1

Git changes from today

- Today's headline change is the new Reviewer and Analyzer surfaces: the workflow-agent catalog now reaches 64 templates, the seed-skill catalog reaches 23 packages, and code review plus deterministic security scanning are now available from both the canvas and the skill layer.

Today's commit appendix 1 of 1

Date	Commit	Subject
2026-05-20	efb8c13	Analyzer and Reviewer agents!!!, by angelahack1

8. Effective Line Inventory by Language

Methodology: tracked text files only. Blank lines and comment-only lines are excluded. Python counts also remove module, class, and function docstrings detected through AST parsing.

Language	Files	Effective lines	Total lines	Share
Python	266	66,989	93,483	66.8%
JavaScript	28	12,902	16,241	12.9%
Markdown	52	11,948	15,367	11.9%
CSS	6	3,362	4,217	3.4%
YAML	68	1,402	2,820	1.4%
HTML	4	823	841	0.8%
Other text	3	789	967	0.8%
Prompt template	2	599	711	0.6%
PowerShell	5	506	752	0.5%
JavaScript module	1	391	458	0.4%
JSON	5	383	383	0.4%
Text	5	150	177	0.1%
Protocol Buffers	1	20	33	0.0%
Batch	1	13	24	0.0%

Total effective lines: 100,277

9. Largest Effective Source Files

Path	Language	Effective	Total
Tlmatini/agent/views.py	Python	7,033	10,128
Tlmatini/agent/tests.py	Python	4,034	5,153
Tlmatini/agent/tools.py	Python	2,201	3,109
BookOfTlmatini.md	Markdown	1,890	2,664
Tlmatini/agent/agents/flowcreator/agentic_skill.md	Markdown	1,854	2,062
Tlmatini/agent/static/agent/css/agentic_control_panel.css	CSS	1,677	2,221
Tlmatini/agent/doc_generation/complete_project_docs.py	Python	1,631	1,844
Tlmatini/agent/static/agent/js/agent_page_init.js	JavaScript	1,360	1,568
Tlmatini/agent/static/agent/js/acp-canvas-core.js	JavaScript	1,284	1,594
Tlmatini/agent/consumers.py	Python	1,240	1,545
README.md	Markdown	1,172	1,646
Tlmatini/agent/static/agent/css/agent_page.css	CSS	1,155	1,380
Tlmatini/agent/static/agent/js/acp-agent-connectors.js	JavaScript	1,145	1,287
Tlmatini/agent/agents/whatstlmatini/whatstlmatini.py	Python	1,059	1,323
Tlmatini/agent/static/agent/js/acp-control-buttons.js	JavaScript	1,036	1,344
Tlmatini/agent/acpx/tests.py	Python	1,035	1,299
Tlmatini/agent/static/agent/js/agent_page_chat.js	JavaScript	1,027	1,420
KIMI.md	Markdown	962	1,156
Tlmatini/agent/agents/teletlmatini/teletlmatini.py	Python	903	1,170
Tlmatini/agent/static/agent/js/canvas_item_dialog.js	JavaScript	898	1,165
Tlmatini/agent/agents/crawler/crawler.py	Python	896	1,285
Tlmatini/agent/static/agent/js/acp-canvas-undo.js	JavaScript	853	964
Tlmatini/agent/acpx/runtime.py	Python	844	1,161
Tlmatini/agent/agents/parametrizer/parametrizer.py	Python	812	1,145
Tlmatini/agent/agents/file_interpreter/file_interpreter.py	Python	764	993

10. Complete Tracked File Tree (Repository Appendix)

Tree appendix 1 of 8

```
Tlamatini/
|-- .gitignore
|-- _acpx_pipeline_demo_report.md
|-- ACPX.md
|-- agents_descriptions.md
|-- BookOfTlamatini.md
|-- build.py
|-- build_installer.py
|-- build_uninstaller.py
|-- CLAUDE.md
|-- CreateShortcut.json
|-- CreateShortcut.ps1
|-- eslint.config.mjs
|-- example_of_prompt_to_make_de-compressor_agent.txt
|-- install.py
|-- KIMI.md
|-- LICENSE
|-- OllamaPricing.png
|-- package.json
|-- pnpm-lock.yaml
|-- prompt_example_2_create_agent.txt
|-- README.md
|-- regen_secrets.py
|-- register_flw.ps1
|-- RemoveShortcut.ps1
|-- requirements.txt
|-- Tlamatini.ico
|-- Tlamatini.jpg
|-- Tlamatini.ps1
|-- tlamatini_app_summary.pdf
|-- Tlamatini_extended_Artificial_Intelligence_Humanly_Tempered.pptx
|-- uninstall.py
|-- unregister_flw.ps1
|-- VERSIONING.md
|-- versioning.py
|-- .codex/
|   |-- skills/
|   |   |-- full-project-pdf-dossier/
|   |   |   |-- SKILL.md
|   |   |-- overlap-safe-pptx-dossier/
|   |   |   |-- SKILL.md
|   |-- .github/
|   |   |-- workflows/
|   |   |   |-- name-guard.yml
|   |-- docs/
|   |   |-- claude/
|   |   |   |-- acpx.md
|   |   |   |-- agents.md
|   |   |   |-- architecture.md
|   |   |   |-- exec-report.md
|   |   |   |-- frontend.md
|   |   |   |-- gotchas.md
|   |   |   |-- INDEX.md
|   |   |   |-- mcp-tools.md
|   |   |   |-- multi-turn.md
|   |-- pyinstaller_hooks/
|   |   |-- hook-numpy.py
|-- Tlamatini/
|   |-- cat_art.py
|   |-- manage.py
|   |-- .agents/
|   |   |-- workflows/
|   |   |   |-- create_new_agent.md
|   |-- .mcps/
|   |   |-- create_new_mcp.md
|   |-- agent/
|   |   |-- __init__.py
|   |   |-- admin.py
|   |   |-- apps.py
|   |   |-- capability_registry.py
|   |   |-- chain_files_search_lcel.py
|   |   |-- chain_system_lcel.py
|   |   |-- chat_agent_registry.py
|   |   |-- chat_agent_runtime.py
|   |   |-- chat_history_loader.py
|   |   |-- command_togen_pb2.txt
```

Tree appendix 2 of 8

```
-- config.json
-- config_loader.py
-- constants.py
-- consumers.py
-- embedding_memory_guard.py
-- filesearch.proto
-- filesearch_pb2.py
-- filesearch_pb2_grpc.py
-- global_execution_planner.py
-- global_state.py
-- gpu_perf.py
-- inet_determiner.py
-- mcp_agent.py
-- mcp_files_search_client.py
-- mcp_files_search_server.py
-- mcp_system_client.py
-- mcp_system_server.py
-- models.py
-- orphan_reaper.py
-- path_guard.py
-- prompt.pmt
-- rag_enhancements.py
-- routing.py
-- test_de_compressor.py
-- test_embedding_memory_guard.py
-- test_flow_contracts.py
-- test_password_quoting.py
-- test_tkinter_availability.py
-- tests.py
-- tools.py
-- urls.py
-- version.py
-- views.py
-- web_search_llm.py
-- acpx/
|   |-- __init__.py
|   |-- agent_registry.py
|   |-- config.py
|   |-- permissions.py
|   |-- runtime.py
|   |-- service.py
|   |-- session_store.py
|   |-- tests.py
|   |-- tools.py
|   |-- windows_spawn.py
-- agents/
|   |-- acpxer/
|   |   |-- acpxer.py
|   |   |-- config.yaml
|   |-- analyzer/
|   |   |-- analyzer.py
|   |   |-- config.yaml
|   |-- and/
|   |   |-- and.py
|   |   |-- config.yaml
|   |-- apirer/
|   |   |-- apirer.py
|   |   |-- config.yaml
|   |-- asker/
|   |   |-- asker.py
|   |   |-- config.yaml
|   |-- barrier/
|   |   |-- barrier.py
|   |   |-- config.yaml
|   |   |-- test_barrier.py
|   |-- cleaner/
|   |   |-- __init__.py
|   |   |-- cleaner.py
|   |   |-- config.yaml
|   |-- counter/
|   |   |-- config.yaml
|   |   |-- counter.py
|   |-- crawler/
|   |   |-- config.yaml
|   |   |-- crawler.py
|   |-- croner/
```

Tree appendix 3 of 8

```

|-- config.yaml
|-- croner.py
-- de_compressor/
|-- config.yaml
|-- de_compressor.py
-- deleter/
|-- config.yaml
|-- deleter.py
-- dockerer/
|-- config.yaml
|-- dockerer.py
-- emailer/
|-- config.yaml
|-- emailer.py
-- ender/
|-- config.yaml
|-- ender.py
-- executer/
|-- config.yaml
|-- executer.py
-- file_creator/
|-- config.yaml
|-- file_creator.py
-- file_extractor/
|-- config.yaml
|-- file_extractor.py
-- file_interpreter/
|-- config.yaml
|-- file_interpreter.py
-- flowbacker/
|-- config.yaml
|-- flowbacker.py
-- flowcreator/
|-- agentic_skill.md
|-- config.yaml
|-- flowcreator.py
-- flowhypervisor/
|-- config.yaml
|-- flowhypervisor.py
|-- hypervisor_alert.wav
|-- monitoring-prompt.pmt
-- forker/
|-- config.yaml
|-- forker.py
-- gateway_relayer/
|-- config.yaml
|-- gateway_relayer.md
|-- gateway_relayer.py
-- gatewayer/
|-- config.yaml
|-- gatewayer.md
|-- gatewayer.py
|-- gatewayer_explanation.md
-- gitter/
|-- config.yaml
|-- gitter.py
-- googler/
|-- config.yaml
|-- googler.py
-- image_interpreter/
|-- config.yaml
|-- image_interpreter.py
-- j_decompiler/
|-- config.yaml
|-- j_decompiler.py
-- jenkinser/
|-- config.yaml
|-- jenkinser.py
-- keyboarder/
|-- config.yaml
|-- keyboarder.py
-- kubernetes/
|-- config.yaml
|-- kubernetes.py
-- kyber_cipher/
|-- config.yaml

```

Tree appendix 4 of 8

```
| | | `-- kyber_cipher.py
| | | -- kyber_decipher/
| | | | -- config.yaml
| | | | `-- kyber_decipher.py
| | | -- kyber_keygen/
| | | | -- config.yaml
| | | | `-- kyber_keygen.py
| | | -- mongoxer/
| | | | -- config.yaml
| | | | `-- mongoxer.py
| | | -- monitor_log/
| | | | -- config.yaml
| | | | `-- monitor_log.py
| | | -- monitor_netstat/
| | | | -- config.yaml
| | | | `-- monitor_netstat.py
| | | -- mouser/
| | | | -- config.yaml
| | | | `-- mouser.py
| | | -- mover/
| | | | -- config.yaml
| | | | `-- mover.py
| | | -- node_manager/
| | | | -- config.yaml
| | | | -- node_manager.md
| | | | -- node_manager.py
| | | | -- nodes_example.json
| | | | `-- nodes_example.yaml
| | | -- notifier/
| | | | -- config.yaml
| | | | `-- notifier.py
| | | -- or/
| | | | -- config.yaml
| | | | `-- or.py
| | | -- parametrizer/
| | | | -- config.yaml
| | | | `-- parametrizer.py
| | | -- prompter/
| | | | -- config.yaml
| | | | `-- prompter.py
| | | -- pser/
| | | | -- config.yaml
| | | | `-- pser.py
| | | -- pythonxer/
| | | | -- config.yaml
| | | | `-- pythonxer.py
| | | -- raiser/
| | | | -- config.yaml
| | | | `-- raiser.py
| | | -- recmailer/
| | | | -- config.yaml
| | | | `-- recmailer.py
| | | -- reviewer/
| | | | -- config.yaml
| | | | `-- reviewer.py
| | | -- scper/
| | | | -- config.yaml
| | | | `-- scper.py
| | | -- shoter/
| | | | -- config.yaml
| | | | `-- shoter.py
| | | -- sleeper/
| | | | -- config.yaml
| | | | `-- sleeper.py
| | | -- sqler/
| | | | -- config.yaml
| | | | `-- sqler.py
| | | -- ssher/
| | | | -- config.yaml
| | | | `-- ssher.py
| | | -- starter/
| | | | -- config.yaml
| | | | `-- starter.py
| | | -- stopper/
| | | | -- config.yaml
| | | | `-- stopper.py
```

Tree appendix 5 of 8

```
-- summarizer/
|   |-- config.yaml
|   |-- summarizer.py
-- telegramer/
|   |-- config.yaml
|   |-- telegramer.py
-- telegramrx/
|   |-- config.yaml
|   |-- telegramrx.py
-- teletlamatini/
|   |-- config.yaml
|   |-- teletlamatini.py
-- unrealer/
|   |-- config.yaml
|   |-- unrealer.py
-- whatsapper/
|   |-- config.yaml
|   |-- whatsapper.py
-- whatstlamatini/
|   |-- config.yaml
|   |-- whatstlamatini.py
-- doc_generation/
|   |-- complete_project_docs.py
|   |-- mardown_to_pdf.py
|   |-- refresh_project_docs.py
|   |-- test_output.pdf
-- images/
|   |-- mockup.jpg
|   |-- TlamatiniAbout.jpg
|   |-- XAIHT-Tlamatini.mp4
-- imaging/
|   |-- converter.py
|   |-- image_interpreter.py
|   |-- screenshot.py
-- management/
|   |-- __init__.py
|   |-- commands/
|   |   |-- acpx_lint.py
|   |   |-- startserver.py
-- migrations/
|   |-- 0001_initial.py
|   |-- 0002_populate_db.py
|   |-- 0003_add_cleaner.py
|   |-- 0004_add_deleter.py
|   |-- 0005_add_executor.py
|   |-- 0006_add_notifier.py
|   |-- 0007_add_stopper.py
|   |-- 0008_add_recmailer.py
|   |-- 0009_add_whatsapper.py
|   |-- 0010_add_pythonxer.py
|   |-- 0011_add_asker.py
|   |-- 0012_repopulate_all_agents.py
|   |-- 0013_add_forker.py
|   |-- 0014_repopulate_all_agents.py
|   |-- 0015_add_shoter.py
|   |-- 0016_repopulate_all_agents.py
|   |-- 0017_add_ssher.py
|   |-- 0018_repopulate_all_agents.py
|   |-- 0019_add_scper.py
|   |-- 0020_repopulate_all_agents.py
|   |-- 0021_add_telegramrx.py
|   |-- 0022_repopulate_all_agents.py
|   |-- 0023_add_mongoxer.py
|   |-- 0024_repopulate_all_agents.py
|   |-- 0025_add_telegramer.py
|   |-- 0026_repopulate_all_agents.py
|   |-- 0027_add_sqler.py
|   |-- 0028_repopulate_all_agents.py
|   |-- 0029_add_prompter.py
|   |-- 0030_repopulate_all_agents.py
|   |-- 0031_add_flowcreator.py
|   |-- 0032_repopulate_all_agents.py
|   |-- 0033_add_gitter.py
|   |-- 0034_add_dockerer.py
|   |-- 0035_add_pser.py
|   |-- 0036_add_kuberneter.py
```

Tree appendix 6 of 8

```
-- 0037_add_apirer.py
-- 0038_add_jenkinser.py
-- 0039_add_crawler.py
-- 0040_add_summarizer.py
-- 0041_add_flowhypervisor.py
-- 0042_add_mouser.py
-- 0043_agentmessage_conversation_user.py
-- 0044_add_counter.py
-- 0045_add_file_interpreter.py
-- 0046_add_image_interpreter.py
-- 0047_add_gatewayer.py
-- 0048_add_gateway_relayer.py
-- 0049_add_node_manager.py
-- 0050_add_file_creator.py
-- 0051_add_file_extractor.py
-- 0052_add_kyber_keygen.py
-- 0053_add_kyber_cipher.py
-- 0054_add_kyber_decipher.py
-- 0055_fix_kyber_cipher_names.py
-- 0056_add_parametrizer.py
-- 0057_add_flowbacker.py
-- 0058_add_barrier.py
-- 0059_add_j_decompiler.py
-- 0060_add_agent_parametrizer_tool.py
-- 0061_add_agent_starter_tool.py
-- 0062_sync_agent_control_tools_and_prompts.py
-- 0063_add_agent_parametrizer_prompt.py
-- 0064_add_chat_agent_run_model.py
-- 0065_add_chat_wrapped_agent_tools.py
-- 0066_add_keyboarder.py
-- 0067_add_multi_turn_demo_prompts.py
-- 0068_add_googler_tool.py
-- 0069_add_googler_agent.py
-- 0070_add_teletlamatini.py
-- 0071_acpx_skills.py
-- 0072_add_acpx_demo_prompts.py
-- 0073_acpx_demo_gemini_uplift.py
-- 0074_simplify_demo_prompts.py
-- 0075_add_acpx_tool_surface.py
-- 0076_add_acpxer.py
-- 0077_add_whatstlamatini.py
-- 0078_add_chat_agent_keyboarder_tool.py
-- 0079_add_chat_agent_mouser_tool.py
-- 0080_add_chat_agent_sleeper_tool.py
-- 0081_add_window_present_and_run_wait_tools.py
-- 0082_add_chat_agent_j_decompiler_tool.py
-- 0083_add_de_compressor.py
-- 0084_add_chat_agent_de_compressor_tool.py
-- 0085_add_unrealer.py
-- 0086_add_chat_agent_unrealer_tool.py
-- 0087_add_unrealer_demo_prompt.py
-- 0088_add_reviewer.py
-- 0089_add_analyzer.py
-- __init__.py
-- monitors/
--   |-- monitor_sql.py
-- opus_client/
--   |-- claude_opus_client.py
--   |-- examples.py
--   |-- README.md
-- rag/
--   |-- __init__.py
--   |-- config.py
--   |-- factory.py
--   |-- interaction.py
--   |-- interface.py
--   |-- loaders.py
--   |-- prompts.py
--   |-- retrieval.py
--   |-- splitters.py
--   |-- utils.py
--   |-- chains/
--     |-- base.py
--     |-- basic.py
--     |-- history_aware.py
--     |-- unified.py
```

Tree appendix 7 of 8

```
-- services/
|  |-- __init__.py
|  |-- agent_contracts.py
|  |-- agent_paths.py
|  |-- answer_analyzer.py
|  |-- filesystem.py
|  |-- flow_compiler.py
|  |-- flow_spec.py
|  |-- response_parser.py
-- skills/
|  |-- __init__.py
|  |-- frontmatter.py
|  |-- harness.py
|  |-- io_contract.py
|  |-- registry.py
-- skills_pkg/
|  |-- meta/
|  |  |-- lint.py
|  |  |-- schema.json
|  |-- acp_router/
|  |  |-- SKILL.md
|  |-- code_review/
|  |  |-- SKILL.md
|  |-- github/
|  |  |-- SKILL.md
|  |-- gmail/
|  |  |-- SKILL.md
|  |-- hello_world/
|  |  |-- SKILL.md
|  |-- jira/
|  |  |-- SKILL.md
|  |-- notion/
|  |  |-- SKILL.md
|  |-- security_audit/
|  |  |-- SKILL.md
|  |-- setup_new_acpx_key/
|  |  |-- SKILL.md
|  |-- skill_creator/
|  |  |-- SKILL.md
|  |  |-- scripts/
|  |  |  |-- quick_validate.py
|  |-- slack/
|  |  |-- SKILL.md
|  |-- summarize/
|  |  |-- SKILL.md
|  |-- tlamatini_allowed_hosts_tighten/
|  |  |-- SKILL.md
|  |-- tlamatini_csrf_exempt_audit/
|  |  |-- SKILL.md
|  |-- tlamatini_exec_report_row_adder/
|  |  |-- SKILL.md
|  |-- tlamatini_flow_from_objective/
|  |  |-- SKILL.md
|  |-- tlamatini_flw_doctor/
|  |  |-- SKILL.md
|  |-- tlamatini_new_acp_agent/
|  |  |-- SKILL.md
|  |-- tlamatini_planner_trace_replay/
|  |  |-- SKILL.md
|  |-- tlamatini_static_version_bumper/
|  |  |-- SKILL.md
|  |-- todoist/
|  |  |-- SKILL.md
|  |-- trello/
|  |  |-- SKILL.md
|  |-- weather/
|  |  |-- SKILL.md
-- static/
|  |-- agent/
|  |  |-- css/
|  |  |  |-- agent_page.css
|  |  |  |-- agentic_control_panel.css
|  |  |  |-- login.css
|  |  |  |-- skills_dialog.css
|  |  |  |-- tools_dialog.css
|  |  |  |-- welcome.css
```


Tree appendix 8 of 8

```

-- img/
|  -- spinner.svg
|  -- Tlamatini.ico
-- js/
|  -- acp-agent-connectors.js
|  -- acp-canvas-core.js
|  -- acp-canvas-undo.js
|  -- acp-control-buttons.js
|  -- acp-file-io.js
|  -- acp-flow-snapshot.js
|  -- acp-globals.js
|  -- acp-layout.js
|  -- acp-parametrizer-dialog.js
|  -- acp-running-state.js
|  -- acp-session.js
|  -- acp-undo-manager.js
|  -- acp-validate.js
|  -- agent_page_canvas.js
|  -- agent_page_chat.js
|  -- agent_page_context.js
|  -- agent_page_dialogs.js
|  -- agent_page_init.js
|  -- agent_page_layout.js
|  -- agent_page_state.js
|  -- agent_page_ui.js
|  -- agentic_control_panel.js
|  -- canvas_item_dialog.js
|  -- chat_page_runtime_poller.js
|  -- contextual_menus.js
|  -- shared-runtime-dialogs.js
|  -- skills_dialog.js
|  -- tools_dialog.js
-- sounds/
|  -- hypervisor_alert.wav
|  -- notification.wav
-- video/
|  -- XAIHT-Tlamatini.mp4
-- telegram/
|  -- AztecTlamatini.txt
|  -- config.yaml
|  -- telegram_receiver.py
-- templates/
|  -- agent/
|  |  -- agent_page.html
|  |  -- agentic_control_panel.html
|  |  -- login.html
|  -- welcome.html
-- DB/
|  -- Older/
|  |  -- README.md
|  -- ToLoad/
|  |  -- README.md
-- jd-cli/
|  -- jd-cli.bat
|  -- jd-cli.jar
-- tlamatini/
|  -- __init__.py
|  -- asgi.py
|  -- context_processors.py
|  -- logging_filters.py
|  -- middleware.py
|  -- settings.py
|  -- urls.py
|  -- wsgi.py

```